The dcrgraph package

Drawing DCR graphs in LaTeX

tilzuck

Version 0.2 – draft, subject to change

Contents

1 Introduction **1**

2 Drawing with TikZ **2**
2.1 A first graph . . . 2
2.2 Events and marking . . . 3
2.3 Relations . . . 4
2.4 Guards and time . . . 6
2.5 Collections . . . 7
2.6 Notations: dcr-solutions, classic, coopis23 . . . 8
2.7 Customisation . . . 10
2.8 Symbols in running text . . . 11

3 Importing XML **12**
3.1 A first import . . . 12
3.2 Options . . . 13
3.3 Customising the imported graph . . . 15
3.4 Taking over the code . . . 16
3.5 What is read . . . 16

4 Reference **17**
4.1 The environment . . . 17
4.2 Events . . . 17
4.3 Collections . . . 18
4.4 Relations . . . 18
4.5 Settings . . . 19
4.6 Colours . . . 20
4.7 Importing: `\includedcrgraph` . . . 20
4.8 Adjusting an imported graph . . . 21
4.9 Symbols in text . . . 21

A Compatibility with the DCR portal and DCR-js **22**

1 Introduction

dcrgraph draws DCR (Dynamic Condition Response) graphs in LaTeX, in one of three notations: `dcr-solutions`, the look of the DCR portal of DCR Solutions (the default); `classic`, the arrows of the literature; `coopis23`, the notation proposed at CoopIS 2023 and used by DCR-js. There are two ways to create a graph:

- **Drawing with Ti*k*Z** (Section 2): you write the events and relations yourself, in a `dcrgraph` environment.
- **Importing XML** (Section 3): you model the graph in the DCR portal or in DCR-js, save it as XML and include the file; the graph looks as in the editor, and can be customised in LaTeX.

Requirements.

- Only the file `dcrgraph.sty` and `\usepackage{dcrgraph}`. Everything else it needs is part of every TeX distribution.
- Graphs drawn with Ti*k*Z work with pdfLaTeX and LuaLaTeX.
- Importing needs **LuaLaTeX**. On Overleaf: upload `dcrgraph.sty` and choose Menu → Compiler → LuaLaTeX.
- Graphs with several relations between the same two events need two LaTeX runs; latexmk and Overleaf do this automatically.

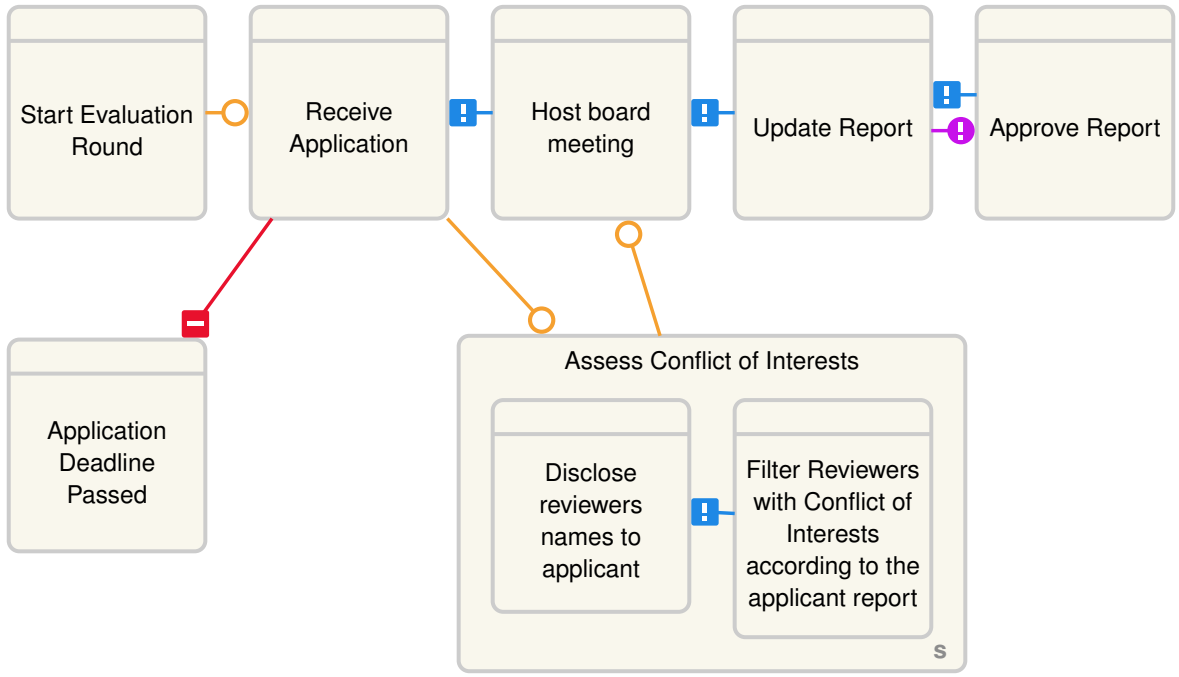
2 Drawing with Ti*k*Z

2.1 A first graph

As a running example we use the evaluation of applications by a board: an evaluation round is started; applications can be received until the deadline has passed; before the board meets, conflicts of interest are assessed (a subprocess); after the meeting the report is updated and then approved.

Everything happens in a `dcrgraph` environment, which is a Ti*k*Z picture. Events come first, then collections (here the subprocess, around its events), then relations:

```
\begin{dcrgraph}
  % events
  \activity{Start Evaluation Round}{start}
  \activity[right=0.6cm of start]{Receive Application}{receive}
  \activity[right=0.6cm of receive]{Host board meeting}{meeting}
  \activity[right=0.6cm of meeting]{Update Report}{update}
  \activity[right=0.6cm of update]{Approve Report}{approve}
  \activity[below=1.6cm of start]{Application Deadline Passed}{deadline}
  \activity[below=2.4cm of meeting]{Disclose reviewers names to applicant}{disclose}
  \activity[below=2.4cm of update]{Filter Reviewers with Conflict of
    Interests according to the applicant report}{filter}
  % a subprocess around two of them
  \subprocess{Assess Conflict of Interests}{assess}{disclose, filter}
  % relations
  \condition{start}{receive}
  \condition{receive}{assess}
  \condition{assess}{meeting}
  \response{receive}{meeting}
  \response{meeting}{update}
  \response{update}{approve}
  \milestone{update}{approve}
  \response{disclose}{filter}
  \exclude{deadline}{receive}
\end{dcrgraph}
```

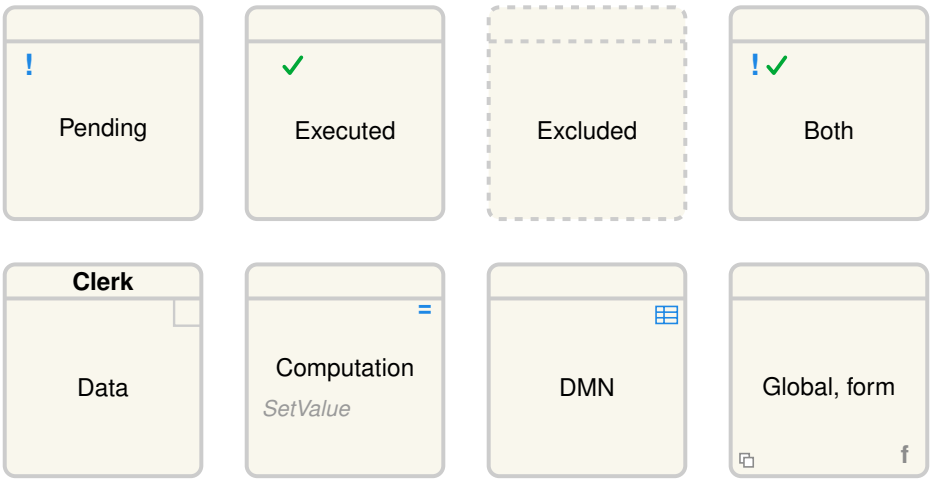


Events are written `\activity[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}`; the ⟨*id*⟩ is used to refer to the event. Relations are written ⟨*relation*⟩`[`⟨*options*⟩`]{`⟨*from*⟩`}{`⟨*to*⟩`}`, always from the source to the target.

2.2 Events and marking

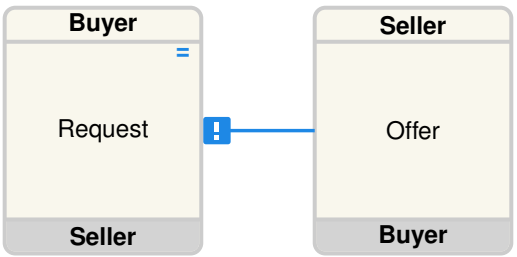
The marking (pending, executed, excluded) and the type of an event are keys of `\activity`; `role=` and `subtitle=` add text.

```
\begin{dcrgraph}
  \activity[pending]{Pending}{a}
  \activity[executed, right=0.6cm of a]{Executed}{b}
  \activity[excluded, right=0.6cm of b]{Excluded}{c}
  \activity[pending, executed, right=0.6cm of c]{Both}{d}
  \activity[role=Clerk, data, below=0.6cm of a]{Data}{e}
  \activity[computation, subtitle=SetValue, right=0.6cm of e]{Computation}{f}
  \activity[dmn, right=0.6cm of f]{DMN}{g}
  \activity[global, form, right=0.6cm of g]{Global, form}{h}
\end{dcrgraph}
```



For choreographies, `receiver=` adds the receiving role(s) in a band at the bottom, as in the DCR portal:

```
\begin{dcrgraph}
  \activity[role=Buyer, receiver=Seller, computation]{Request}{a}
  \activity[role=Seller, receiver=Buyer, right=1.5cm of a]{Offer}{b}
  \response{a}{b}
\end{dcrgraph}
```

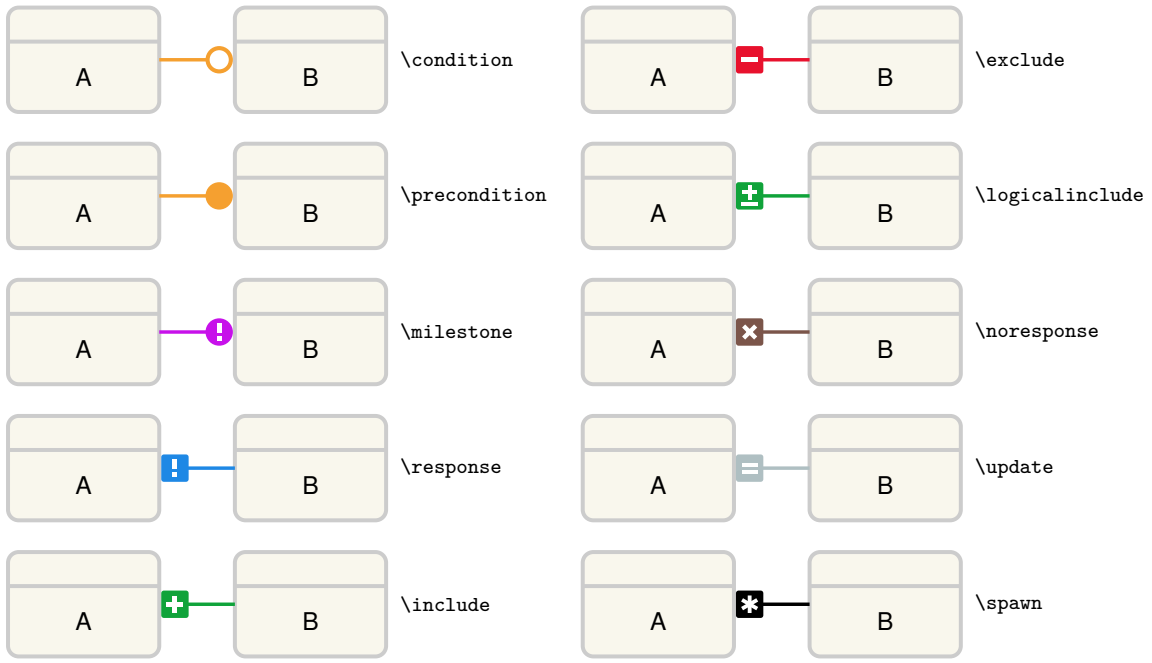


Events are placed with Ti*k*Z: relative to others (`right=1.5cm of a`, `below=2cm of a`) or at coordinates (`at={(4,0)}`). Their size is `minimum width=`, `minimum height=` (default 2.6 cm × 2.8 cm).

2.3 Relations

All relations at a glance (dcr-solutions notation; the marker sits at the target for conditions, preconditions and milestones, at the source for all others):

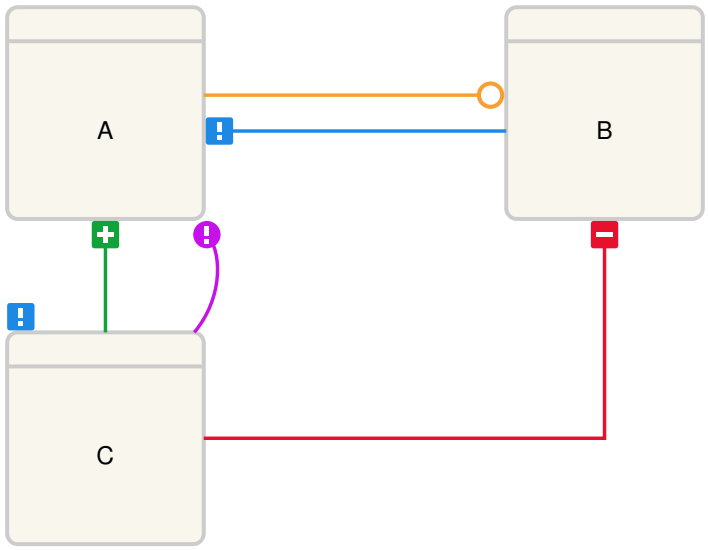
```
\begin{dcrgraph}[small/.style={minimum width=2cm, minimum height=1.3cm},
                 relname/.style={anchor=west, font=\ttfamily\scriptsize}]
  \foreach \rel [count=\i] in {condition, precondition, milestone,
                               response, include} {
    \activity[small, at={(0,-1.8*\i)}]{A}{a\i}
    \activity[small, at={(3,-1.8*\i)}]{B}{b\i}
    \csname\rel\endcsname{a\i}{b\i}
    \node[relname] at (4.05,-1.8*\i) {\textbackslash\rel};
  }
  \foreach \rel [count=\i] in {exclude, logicalinclude, noresponse,
                               update, spawn} {
    \activity[small, at={(7.6,-1.8*\i)}]{A}{c\i}
    \activity[small, at={(10.6,-1.8*\i)}]{B}{d\i}
    \csname\rel\endcsname{c\i}{d\i}
    \node[relname] at (11.65,-1.8*\i) {\textbackslash\rel};
  }
\end{dcrgraph}
```



Placing the lines.

A relation is a straight line between the two boxes. Several relations between the same two events are spread out side by side. A line can start and end at a given point of a box (`a.south`), run through points (`via=`, which keeps right angles), or take any Ti*k*Z option. A relation from an event to itself is a marker on its top edge.

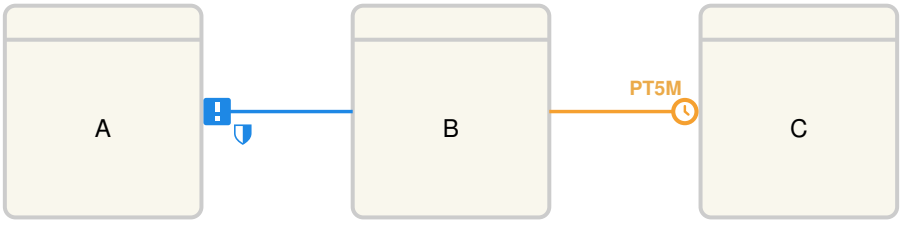
```
\begin{dcrgraph}
  \activity{A}{a}
  \activity[right=4cm of a]{B}{b}
  \activity[below=1.5cm of a]{C}{c}
  \condition{a}{b}                          % two relations between A and B:
  \response{a}{b}                           % spread out automatically
  \include{a.south}{c.north}                % from an anchor to an anchor
  \exclude[via={(b.south |- c.east)}]{b}{c} % right-angled route
  \milestone[bend right=40]{c}{a}           % any TikZ option
  \response{c}{c}                           % self-relation: marker on top
\end{dcrgraph}
```



2.4 Guards and time

A relation can have a guard (`guard={`⟨*expression*⟩`}`). A condition (and a pre-condition) can have a delay (`delay=`⟨*time*⟩), a response a deadline (`deadline=`⟨*time*⟩); other relations have no time (the wrong key is an error). In the dcr-solutions notation they are shown as markers: a shield for a guard, a clock for a time. Their values can be written as text, for the whole graph or for single relations:

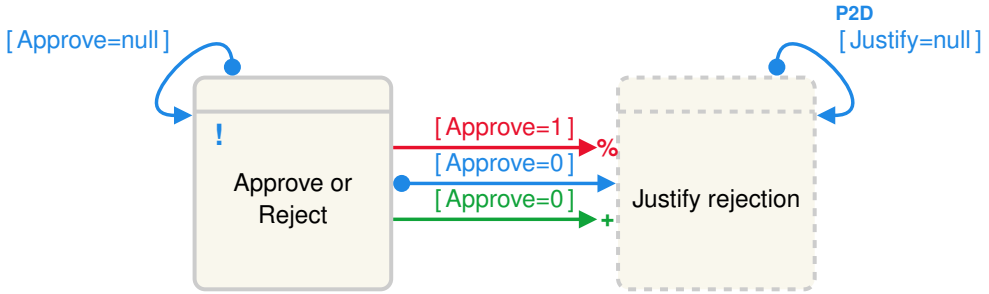
```
\begin{dcrgraph}
  \activity{A}{a}
  \activity[right=2cm of a]{B}{b}
  \activity[right=2cm of b]{C}{c}
  \response[guard={x > 10}]{a}{b}                 % shield
  \condition[delay=PT5M, explicit=delay]{b}{c}    % clock and its time
\end{dcrgraph}
```



- For the graph: `guards=`, `delays=`, `deadlines=` with `explicit` (the value as text), `none` (nothing at all, not even the marker), or the default (`shield`, `marker`); `explicit` for all three.
- For one relation: `explicit=guard`, `explicit={delay,guard}`, `explicit` (all).

In the classic and coopis23 notations there are no shields or clocks: there the values are written as text by default, and `none` leaves them out. A relation from an event to itself shows its values too: next to its loop, away from the event (classic, coopis23), or above its marker (dcr-solutions). A loop goes round the corner given with `corner=`, otherwise round the next free corner. The text size is the style `guard label` (guards) and `delay label` (times):

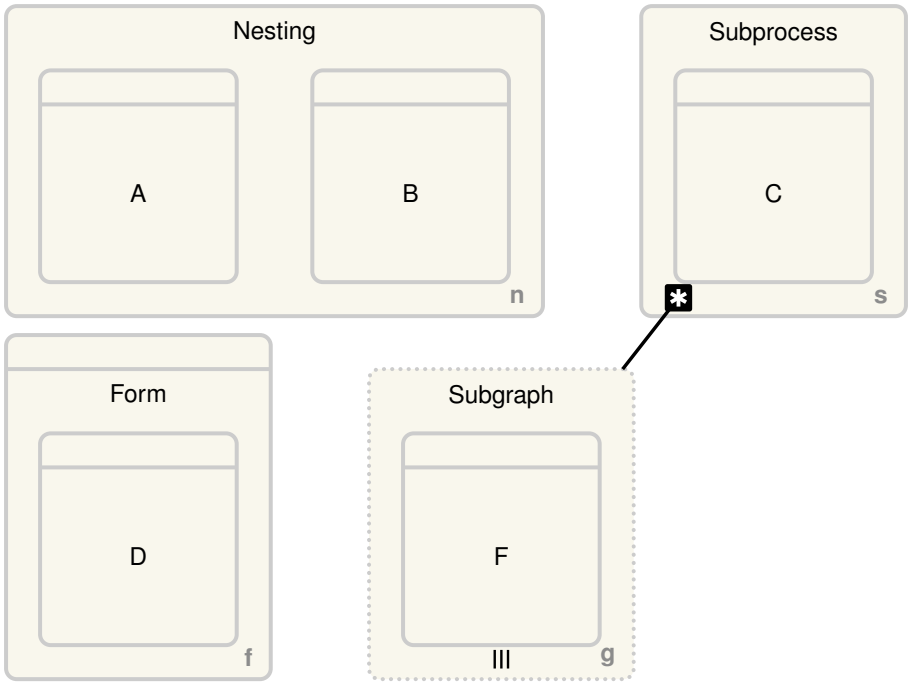
```
\begin{dcrgraph}[notation=classic,
                 guard label/.append style={font=\sffamily\small}]
  \activity[pending]{Approve or Reject}{a}
  \activity[right=3cm of a, excluded]{Justify rejection}{j}
  \exclude[guard={Approve=1}]{a}{j}
  \response[guard={Approve=0}]{a}{j}
  \include[guard={Approve=0}]{a}{j}
  \response[guard={Approve=null}, corner=north west]{a}{a}  % loop top left
  \response[guard={Justify=null}, deadline=P2D]{j}{j}        % next free corner
\end{dcrgraph}
```



2.5 Collections

Nestings, subprocesses, forms and subgraphs group events. They are written after their events and placed around them; relations can start and end at a collection like at an event.

```
\begin{dcrgraph}
  \activity{A}{a}
  \activity[right=1cm of a]{B}{b}
  \nesting{Nesting}{n}{a, b}
  \activity[right=2.2cm of b]{C}{c}
  \subprocess{Subprocess}{s}{c}
  \activity[below=2cm of a]{D}{d}
  \form{Form}{f}{d}
  \activity[right=2.2cm of d]{F}{fx}
  \subgraph{Subgraph}{g}{fx}
  \spawn{c}{g}
\end{dcrgraph}
```



2.6 Notations: dcr-solutions, classic, coopis23

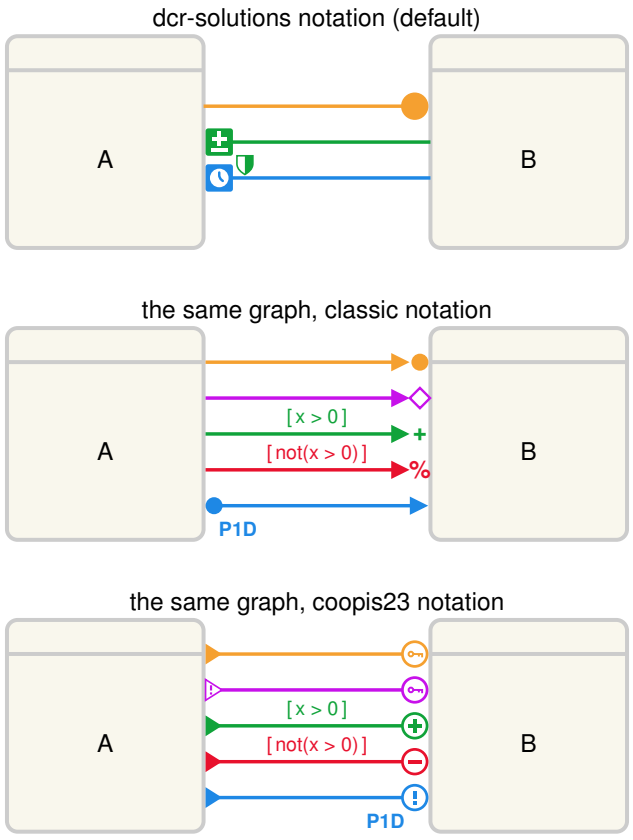
The relations can be drawn in three notations (events and collections look the same in all of them):

- `dcr-solutions` (default): the markers of the DCR portal.
- `classic`: the arrows of the literature.
- `coopis23`: the notation proposed in *Designing Semantically Transparent Declarative Process Constraints* (CoopIS 2023) and used by DCR-js: a triangle at the source and a circle with the symbol at the target (a key for condition and milestone; the milestone's triangle is hollow, with a "!").

```
\textsf{\small dcr-solutions notation (default)}\par
\begin{dcrgraph}
  \activity{A}{a}
  \activity[right=3cm of a]{B}{b}
  \precondition{a}{b}
  \logicalinclude[guard={x > 0}]{a}{b}
  \response[deadline=P1D]{a}{b}
\end{dcrgraph}

\bigskip
\textsf{\small the same graph, classic notation}\par
\begin{dcrgraph}[notation=classic]
  \activity{A}{a}
  \activity[right=3cm of a]{B}{b}
  \precondition{a}{b}
  \logicalinclude[guard={x > 0}]{a}{b}
  \response[deadline=P1D]{a}{b}
\end{dcrgraph}

\bigskip
\textsf{\small the same graph, coopis23 notation}\par
\begin{dcrgraph}[notation=coopis23]
  \activity{A}{a}
  \activity[right=3cm of a]{B}{b}
  \precondition{a}{b}
  \logicalinclude[guard={x > 0}]{a}{b}
  \response[deadline=P1D]{a}{b}
\end{dcrgraph}
```



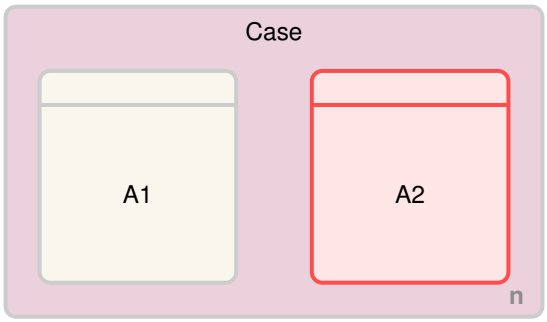
The classic and coopis23 notations have no symbols for two relations of the DCR portal; they

are drawn as what they mean: a pre-condition as a condition plus a milestone (a time on the condition only), a logical include with the guard g as an include with the guard g plus an exclude with the guard `not(`g`)` (format: `\dcrnotguard`). Update, no-response and spawn are not part of the coopis23 proposal; there they get their dcr-solutions symbol in the circle. The notation of a whole document: `\usepackage[classic]{dcrgraph}` or `\usepackage[coopis23]{dcrgraph}`; of one graph: `notation=classic` or `notation=coopis23`.

2.7 Customisation

Colours. `bgcolor`, `bordercolor` and `textcolor` take any xcolor colour; own colours are defined with `\definecolor`.

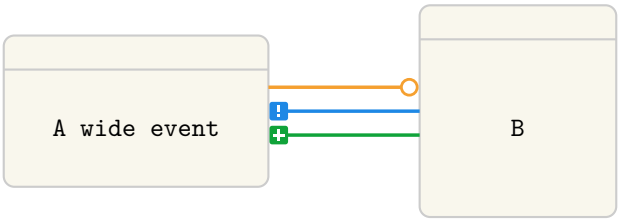
```
\definecolor{casepink}{HTML}{EAD1DC}
\begin{dcrgraph}
  \activity{A1}{a1}
  \activity[right=1cm of a1, bgcolor=red!10, bordercolor=red!70]{A2}{a2}
  \nesting[bgcolor=casepink]{Case}{case}{a1, a2}
\end{dcrgraph}
```



The defaults of a whole document: `\dcrset{bgcolor=..., nesting bgcolor=...}`, `\colorlet{dcr border}{...}`; the colours of the relations: `\colorlet{dcr condition}{...}` and so on (Section 4.6).

Sizes and fonts. Markers, the spacing of parallel lines, line widths and the font are settings of the graph or the document (Section 4.5); boxes have any size.

```
\begin{dcrgraph}[marker size=7pt, parallel distance=9pt,
                 border width=0.8pt, font=\ttfamily]
  \activity[minimum width=3.5cm, minimum height=2cm]{A wide event}{a}
  \activity[right=2cm of a]{B}{b}
  \condition{a}{b}
  \response{a}{b}
  \include{a}{b}
\end{dcrgraph}
```



2.8 Symbols in running text

For papers, every relation has a command for running text and formulas, `\dcr`⟨*relation*⟩`[`⟨*options*⟩`]`. It draws the relation as in a graph, in the current notation, with the same options (guards, times, notation):

```
The relation $A \dcrresponse B$ makes $B$ pending if $A$ is executed,
while $A \dcrcondition B$ expresses that $A$ must have been executed
before $B$ can be executed. Both of these relations can be timed, by
using the \texttt{delay} or \texttt{deadline} option.
$A \dcrcondition[delay=5d] B$ expresses that $A$ must have been executed
5 days before $B$ can be executed, and $A \dcrresponse[deadline=5d] B$
states that $B$ must be executed within 5 days.

The other available inline relations are $A \dcrinclude B$ and
$A \dcrexclude B$ ($A$ includes or excludes $B$), $A \dcrmilestone B$
($B$ cannot be executed while $A$ is pending), $A \dcrprecondition B$
(a condition and a milestone in one), $A \dcrlogicalinclude B$,
$A \dcrupdate B$ ($A$ updates the value of $B$), $A \dcrnoresponse B$
($A$ removes the pending of $B$) and $A \dcrspawn B$ ($A$ spawns $B$).

Every relation can have a guard, written above the line with the
\texttt{guard} option: $A \dcrinclude[guard={x > 5}] B$ includes $B$
only if $x > 5$. Guard and time can be combined:
$A \dcrresponse[guard=ok, deadline=5d] B$.
```

The relation A —! B makes B pending if A is executed, while A —○ B expresses that A must have been executed before B can be executed. Both of these relations can be timed, by using the `delay` or `deadline` option. A —5d○ B expresses that A must have been executed 5 days before B can be executed, and A 5d— B states that B must be executed within 5 days.

The other available inline relations are A —+ B and A —− B (A includes or excludes B), A —! B (B cannot be executed while A is pending), A —● B (a condition and a milestone in one), A —± B, A —= B (A updates the value of B), A —× B (A removes the pending of B) and A —* B (A spawns B).

Every relation can have a guard, written above the line with the `guard` option: A —[x > 5]+ B includes B only if $x > 5$. Guard and time can be combined: A 5d—[ok] B.

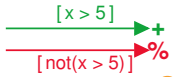
Guards are written above the line, times below it. The notation of a symbol, the symbols in running text and in sub- and superscripts, and the length of the line:

```
By default the symbols use the notation of the document; the
\texttt{notation} option changes it for one symbol. The same
condition in the three notations: $A \dcrcondition B$,
$A \dcrcondition[notation=classic] B$ and
$A \dcrcondition[notation=coopis23] B$.

The classic and coopis23 notations have no logical include: it is
drawn as an include with the guard and an exclude with the negated
guard, $A \dcrlogicalinclude[notation=classic, guard={x > 5}] B$.
A pre-condition becomes a condition and a milestone:
$A \dcrprecondition[notation=coopis23, delay=5d] B$.

The symbols also work without math mode, as in A \dcrresponse B, and
shrink in sub- and superscripts like $\rightarrow$:
$X_{A \dcrcondition B}$, $Y^{A \dcrinclude[guard=g] B}$.

With \texttt{length}, a line is longer than its minimum:
$A \dcrexclude[length=4em] B$.
```

By default the symbols use the notation of the document; the `notation` option changes it for one symbol. The same condition in the three notations: A —○ B, A →● B and A ▶—⊶ B. The classic and coopis23 notations have no logical include: it is drawn as an include with the guard and an exclude with the negated guard, A 

B. A pre-condition becomes a condition and a milestone: A 

B.
The symbols also work without math mode, as in A  B, and shrink in sub- and superscripts like →: $X_{A \dcrcondition B}$, $Y^{A \dcrinclude[guard=g] B}$.
With `length`, a line is longer than its minimum: A  B.

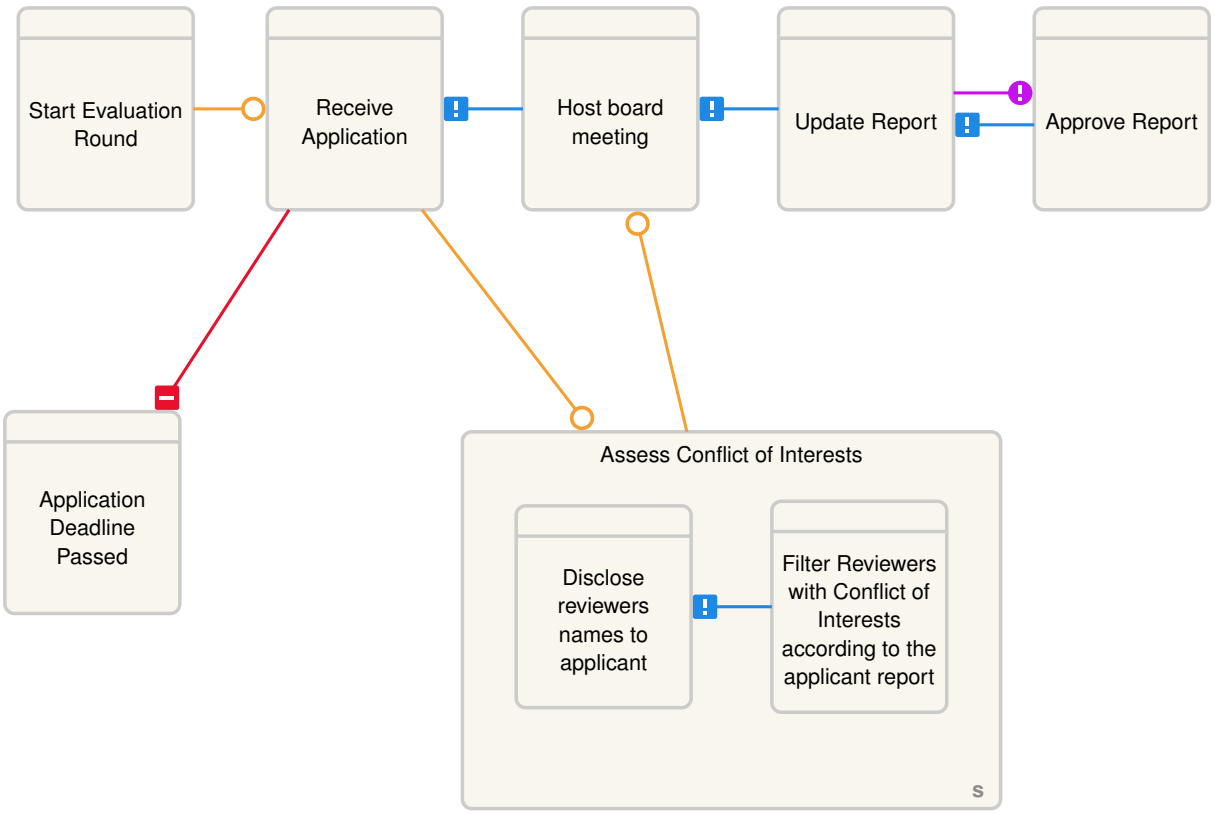
All commands and options: Section 4.9.

3 Importing XML

3.1 A first import

The running example of Section 2, the evaluation of applications, modelled in DCR-js and exported as XML. Put the file next to your document and include it:

```
\resizebox{\linewidth}{!}{%
  \includedcrgraph{examples/application process.xml}}
```

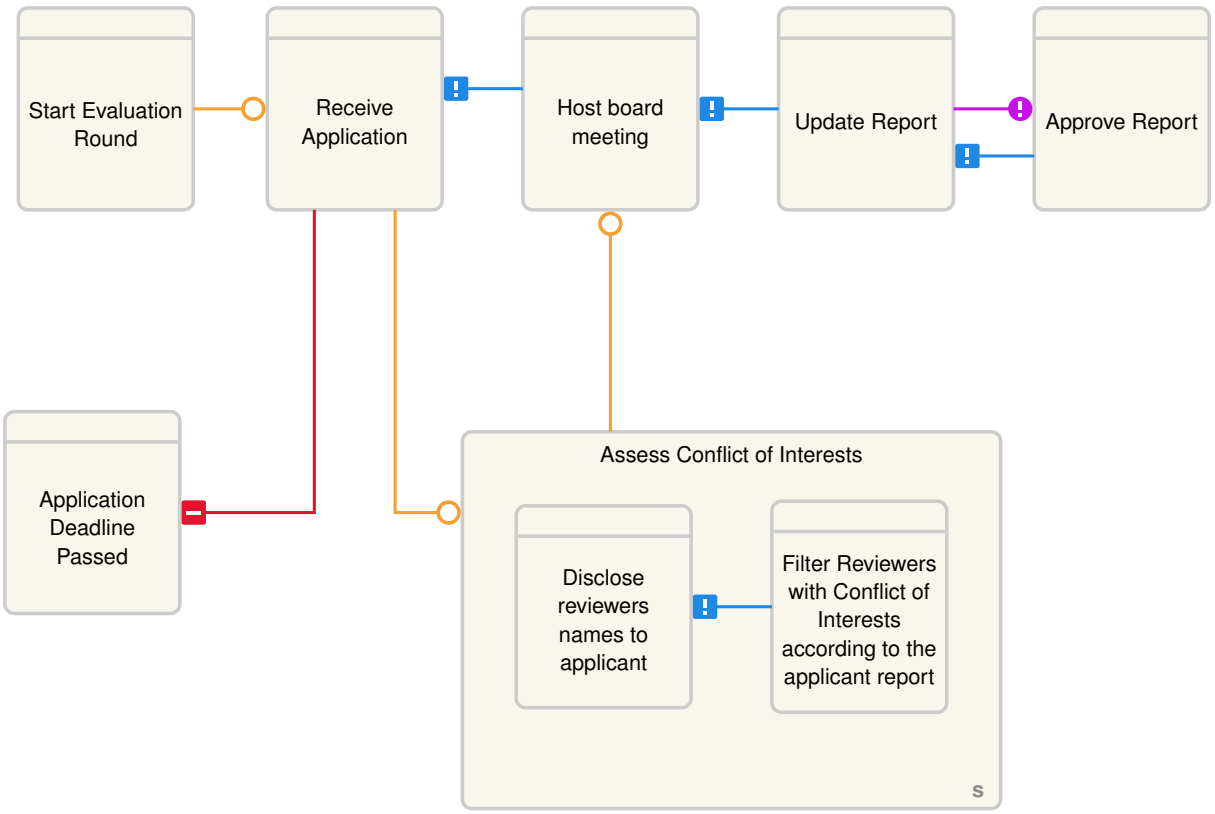


The graph looks as in the editor: positions and sizes, colours, roles, marking, all events, collections and relations with their markers. `\includedcrgraph` draws an ordinary `dcrgraph` picture, so everything in Section 2 applies to it too: the notation, the display of guards and times, the settings.

3.2 Options

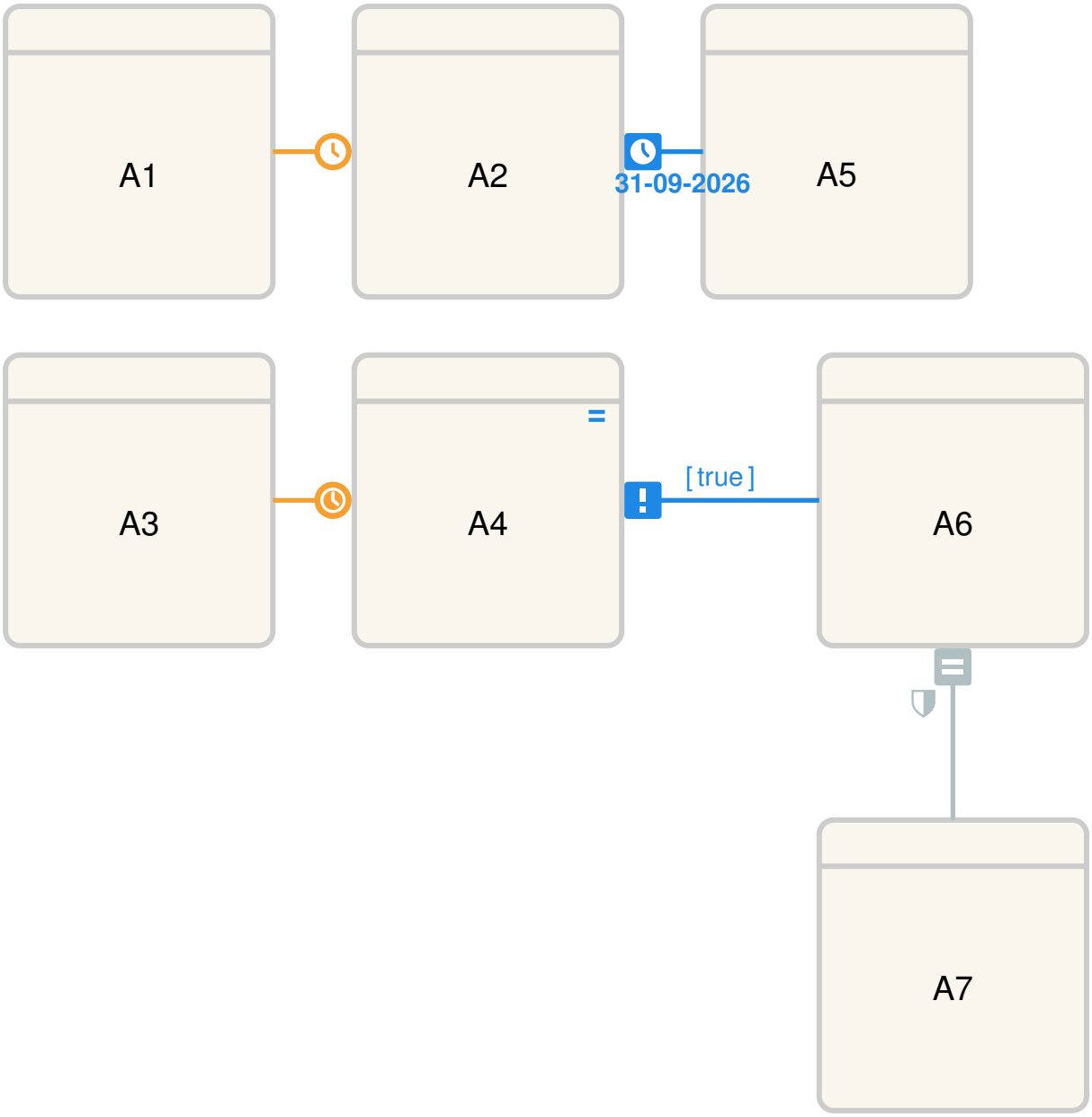
The options of `\includedcrgraph` (Section 4.7) choose the notation (`notation=classic`), how guards and times are shown, and a few things of the import itself. Relations are drawn as straight lines by default; files from DCR-js also store the routes of the lines, which `waypoints` uses:

```
\resizebox{\linewidth}{!}{%
  \includedcrgraph[waypoints]{examples/application process.xml}}
```



Guards, delays and deadlines can also be shown for single relations, named by their ids in the file as "⟨*source*⟩ ⟨*type*⟩ ⟨*target*⟩" (or "⟨*source*⟩ ⟨*target*⟩" if there is only one relation between the two; `value` is the update relation):

```
\resizebox{\linewidth}{!}{%
  \includedcrgraph[guards={A4 response A6}, deadlines=explicit]
    {../../tests/xml/dcr-solutions/04-timed-guarded.xml}}
```

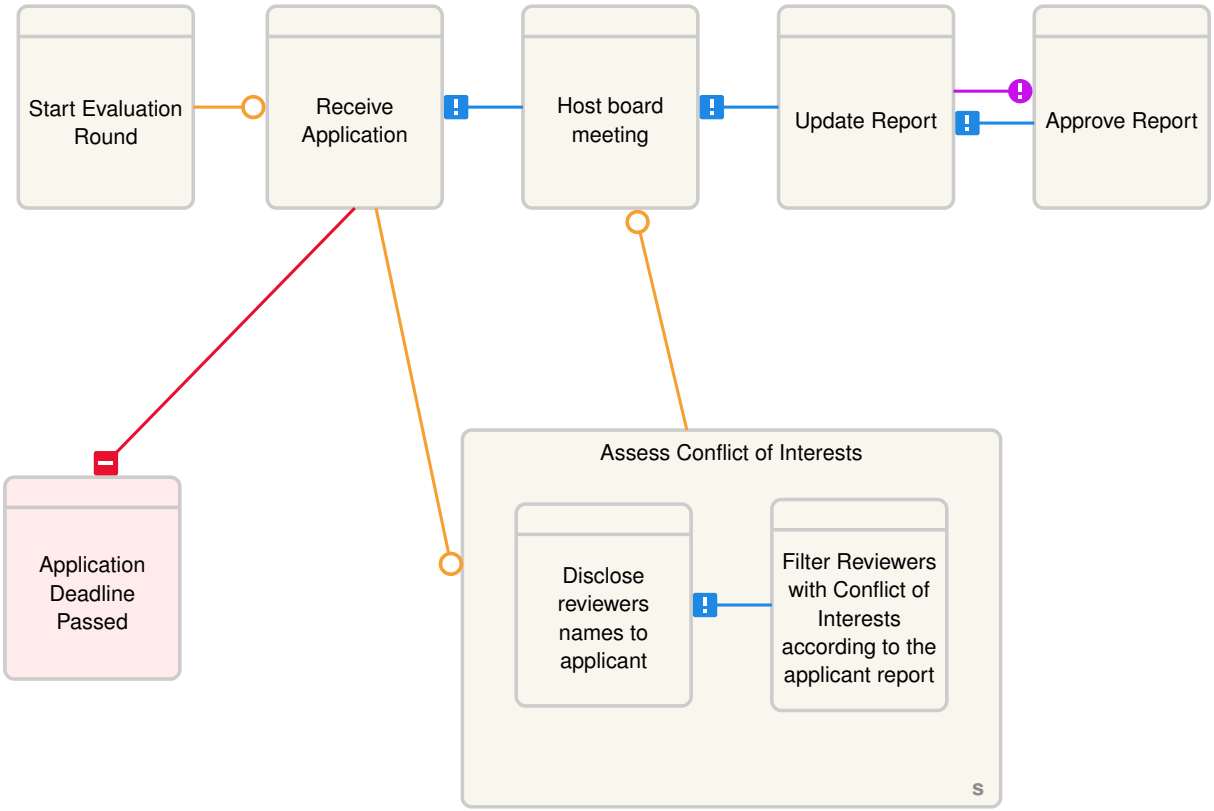


A name that matches no relation, or a relation without such a value, gives a warning.

3.3 Customising the imported graph

The imported graph can be changed in LaTeX without touching the file: put the changes into a `dcrgraphfile` environment. Events and relations are named by their ids in the file (DCR-js generates ids like `Event_191im4m`; the DCR portal uses the ids given in the editor).

```
\resizebox{\linewidth}{!}{%
\begin{dcrgraphfile}{examples/application process.xml}
  % the exclude: from the top of "Deadline Passed" to the bottom of "Receive"
  \adjustrelation{Event_1upi9ve exclude Event_191im4m}{from=north, to=south}
  % the condition into the subprocess: along the left
  \adjustrelation{Event_191im4m condition SubProcess_1fhz1v1}{to=west}
  % "Deadline Passed" a little lower, and highlighted
  \adjustactivity{Event_1upi9ve}{yshift=-1cm, bgcolor=red!8}
\end{dcrgraphfile}}
```



- `\adjustrelation{`⟨*relation*⟩`}{`⟨*options*⟩`}`: `from=` and `to=` set the point of the box where the line starts and ends (`north`, `south`, `east`, `west`, `south east`, …, or an angle like `-60`); any other option of a relation works too: `bend left=20`, `via={(5,-11)}`, `explicit=guard`.
- `\adjustactivity{`⟨*id*⟩`}{`⟨*options*⟩`}`: `shift={(x,y)}`, `xshift=`, `yshift=` move a box (a collection with everything inside it, and the collections around it grow); any other option of an activity works too (colours, `subtitle=`, …).

When the graph is changed in the portal and exported again, the changes still apply; one that names something no longer in the file gives a warning.

3.4 Taking over the code

With `export=`⟨*file.tex*⟩, `\includedcrgraph` also writes the `dcrgraph` code of the graph into a file (and into the `.log`). Include it with `\input` instead of the XML, and edit it freely; it also works with pdfLaTeX. On Overleaf the file is under “Logs and output files”; copy it into your project.

3.5 What is read

Two formats are read; which one a file has is recognised automatically:

- **DCR Solutions XML**, the format of the DCR portal (DCR-js can export it too): events with labels, roles, receivers (the portal's read roles), types, marking and colours; nestings, subprocesses, forms and subgraphs (with their own events); all relations with their guards and times.
- **DCR-js XML**, the format DCR-js saves its graphs in: events with labels, roles and marking; nestings and subprocesses; the relations DCR-js has (condition, response, include, exclude, milestone, update, spawn, no-response). DCR-js has no guards, times, pre-conditions, logical includes, event types or colours.

The positions are taken from the file; the routes of the lines only with `waypoints` (otherwise relations are straight lines; every line gets its own place on the edges of its two boxes, evenly spaced with the other lines there, so no two lines or markers lie on top of each other, and no second LaTeX run is needed). Things the package cannot draw are skipped with a warning. Details: Appendix A.

4 Reference

4.1 The environment

`\begin{dcrgraph}[`⟨*options*⟩`]`	A Ti*k*Z picture; options: any Ti*k*Z option, and the settings of Section 4.5 (e.g. `notation=classic`). A setting wins over a Ti*k*Z key of the same name (`font=`); the prefix `dcr/` (`dcr/notation=classic`) still works.

4.2 Events

`\activity[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}`

Key	Meaning
`role=`⟨*text*⟩	role in the header; several: `role={A, B}`
`receiver=`⟨*text*⟩	choreographies: receiver in a grey band at the bottom (the box is one header taller); several: `receiver={A, B}`
`subtitle=`⟨*text*⟩	grey italic text below the label, cut off with …
`pending`	blue "!" top left
`executed`	green tick top left
`excluded`, `included`	dashed outline / normal (default)
`data`	data event: page icon in the top right corner
`computation`	computation event: blue "=" top right
`dmn`	DMN event (computation by decision table): blue table grid
`global`	global event: two squares bottom left
`access`	access control: grey "#" top right
`form`	event with a form: "f" bottom right
`subgraph`	subgraph (template) event, target of spawns: dotted, "|||", "g"
`bgcolor=`, `bordercolor=`, `textcolor=`	colours of this event
anything else	passed to the Ti*k*Z node: `at={(x,y)}`, `right=2cm of a`, `minimum width=3cm`, …

The label is wrapped to the width of the box; a box grows taller (evenly around its centre) when its label needs more room than it has, so long labels never run out of the box.

4.3 Collections

Written after their events; ⟨*children*⟩ is a list of ids.

Command	Meaning
`\nesting[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}{`⟨*children*⟩`}`	nesting, “n”
`\subprocess[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}{`⟨*children*⟩`}`	subprocess, “s”
`\form[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}{`⟨*children*⟩`}`	form, “f”
`\subgraph[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*id*⟩`}{`⟨*children*⟩`}`	subgraph with its own events: dotted, “|||”, “g”
Keys	`role=`, `subtitle=`, `pending`, `executed`, `excluded`, `bgcolor=`, `bordercolor=`, `textcolor=`; `area={(x1,y1)(x2,y2)}`: exact box instead of fitting the children

4.4 Relations

⟨*relation*⟩`[`⟨*options*⟩`]{`⟨*from*⟩`}{`⟨*to*⟩`}`

Command	Colour	Marker (dcr-solutions notation)
`\condition`	orange	open circle at the target
`\precondition`	orange	filled dot at the target
`\milestone`	purple	circled “!” at the target
`\response`	blue	“!” square at the source
`\include`	green	“+” square at the source
`\exclude`	red	“−” square at the source
`\logicalinclude`	green	“±” square at the source
`\noresponse`	brown	“×” square at the source
`\update`	grey	“=” square at the source (the “value” relation)
`\spawn`	black	“∗” square at the source (to a subgraph)

Option	Meaning
`delay=`⟨*time*⟩	delay of a condition or pre-condition: clock marker
`deadline=`⟨*time*⟩	deadline of a response: clock marker (no other relation has a time)
`guard`, `guard={`⟨*expr*⟩`}`	guard: shield, or the expression
`explicit={guard,delay,deadline}`	write these values of this relation as text; `explicit` alone: all
`via={(x1,y1),(x2,y2)}`	route the line through these points
`selfpos=`⟨*length*⟩	for a relation from an event to itself: where its marker sits on the top edge (its centre, from the left edge); without it, the markers are lined up from the left corner (`\includedcrgraph` sets it, keeping clear of other relations)
`corner=`⟨*corner*⟩	classic, coopis23: for a relation from an event to itself, the corner of its loop (`north east`, `north west`, `south east`, `south west`); without it, the next corner no other loop of the event uses
`bundle=`⟨*node*⟩`:`⟨*name*⟩	spread this relation together with the others of the same bundle, as if they were parallel; for relations with a common end ⟨*node*⟩ whose lines or markers would lie on top of each other (`\includedcrgraph` sets this automatically)
anything else	Ti*k*Z: `bend left=20`, `out=90`, ...

A relation from an event to itself is drawn as a marker above the event (dcr-solutions) or a loop round a corner (classic, coopis23); its guard and time, if shown as text, are written above the marker or next to the loop. Several relations between the same two events are spread out automatically, unless they have Ti*k*Z options.

4.5 Settings

With `\dcrset{`⟨*settings*⟩`}` for the document, or in the options of one graph: `\begin{dcrgraph}[notation=classic, explicit]`. Styles are changed the same way: `guard label/.append style={font=\sffamily\small}`.

Setting	Meaning (default)
`notation=dcr-solutions|classic|coopis23`	markers of the DCR portal, arrows, or triangle and circle (default: dcr-solutions)
`guards=shield|explicit|both|none`	how guards are shown
`delays=marker|explicit|none`	times of conditions etc.
`deadlines=marker|explicit|none`	times of responses
`explicit`	all three explicit
Defaults: dcr-solutions shows only the markers (shield, clock); classic and coopis23 have no shields or clocks and show the values as text. `explicit` adds the text (dcr-solutions), `none` shows nothing at all (all notations).	
`font=`	font of the graph (`\sffamily`)
`subtitle font=`	font of subtitles
`bgcolor=`, `nesting bgcolor=`	default backgrounds
`header height=`	height of the header (0.45 cm)
`border width=`	outline of boxes (1.5 pt)
`relation width=`	relation lines (1.3 pt)
`marker size=`	markers (10.3 pt); separately: `condition marker size=`, `response marker size=`
`marker gap=`	gap between marker and box (0 pt)
`parallel distance=`	distance of parallel relations (13.5 pt)
`classic marker size=`	symbols in the classic notation
`coopis bend=`	coopis23: with a radius, a slanted line leaves the triangle straight out and turns into its direction with a bend of this radius (0 pt: straight lines)
`data size=`, `dmn size=`, `guard size=`, `tick size=`	sizes of the icons
`guard label` (style)	guards as text (`\scriptsize`); change with `guard label/.append style={...}`
`delay label` (style)	delays and deadlines as text (bold `\scriptsize`)

Package options: `\usepackage[classic]{dcrgraph}`, `\usepackage[coopis23]{dcrgraph}` (or `dcr-solutions`, the default).

4.6 Colours

Change with `\definecolor` or `\colorlet`, e.g. `\colorlet{dcr condition}{orange}`.

Colour	Used for
`dcr activity`, `dcr nesting`	backgrounds (#F9F7ED)
`dcr border`, `dcr text`	outlines (#CCCCCC), labels (black)
`dcr condition`, `dcr precondition`, `dcr response`, `dcr include`, `dcr exclude`, `dcr logical include`, `dcr milestone`, `dcr noresponse`, `dcr update`, `dcr spawn`	relations
`dcr executed`, `dcr delay`, `dcr subtitle`, `dcr icon`, `dcr dmn`	markers, labels and icons
`dcr receiver`	receiver band (choreographies)
`dcr subgraph`, `dcr subgraph collection`	subgraphs

4.7 Importing: `\includedcrgraph`

`\includedcrgraph[`⟨*options*⟩`]{`⟨*file*⟩`}` (LuaLaTeX)

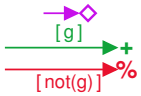
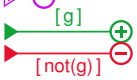
Option	Meaning
`waypoints`	draw relations along the routes stored in the file (default: straight lines)
`notation=classic|coopis23`	the notation (default dcr-solutions)
`guards=`, `delays=`, `deadlines=`	as the settings above, or a list of relations: `guards={A4 response A6, A6 A7}`; each entry "⟨*source*⟩ ⟨*type*⟩ ⟨*target*⟩" or "⟨*source*⟩ ⟨*target*⟩" (ids from the file; `value` = `\update`)
`explicit`	all values as text
`marking=false`	without executed / pending / excluded
`scale=`⟨*factor*⟩	scale the positions (not the boxes)
`export=`⟨*file.tex*⟩	also write the generated code to a file, and into the `.log`
`format=`	not needed: the format (DCR Solutions XML or DCR-js XML) is recognised from the file; accepted values `auto`, `dcrsolutions`, `dcrjs`

4.8 Adjusting an imported graph

Command	Meaning
`\begin{dcrgraphfile}[`⟨*options*⟩`]{`⟨*file*⟩`}`	an imported graph with adjustments; options as for `\includedcrgraph`; drawn at `\end{dcrgraphfile}` (LuaLaTeX)
`\adjustactivity{`⟨*id*⟩`}{`⟨*options*⟩`}`	`shift={(x,y)}`, `xshift=`, `yshift=` (in cm, or with a unit): move the box, a collection with everything inside it; other options: as for `\activity`
`\adjustrelation{`⟨*relation*⟩`}{`⟨*options*⟩`}`	⟨*relation*⟩: "⟨*source*⟩ ⟨*type*⟩ ⟨*target*⟩" or "⟨*source*⟩ ⟨*target*⟩"; `from=`, `to=`: anchor of the box where the line starts/ends; other options: as for relations

4.9 Symbols in text

`\dcr`⟨*relation*⟩`[`⟨*options*⟩`]`, in text (`A \dcrcondition B`) or in math (`$A \dcrcondition B$`, spaced as a relation symbol). The symbol is the relation of a graph, drawn from one point to another, so it follows the notation and the colours of the document.

Command	dcr-solutions	classic	coopis23
`\dcrcondition`			
`\dcrresponse`			
`\dcrinclude`			
`\dcrexclude`			
`\dcrmilestone`			
`\dcrupdate`			
`\dcrnoresponse`			
`\dcrspawn`			
`\dcrprecondition`			
`\dcrlogicalinclude`			

Key	Meaning
`guard=`⟨*expr*⟩	the guard, above the line (the line grows to fit it); classic and coopis23: the negated guard of a logical include below
`delay=`⟨*time*⟩	the delay of `\dcrcondition` and `\dcrprecondition`, below the line; dcr-solutions also shows the clock marker
`deadline=`⟨*time*⟩	the deadline of `\dcrresponse`, the same (no other relation has a time)
`notation=`	`dcr-solutions`, `classic` or `coopis23`; default: the notation of the document
`length=`	shortest length of the line (default 1.8em)

As with `\xrightarrow`, guard and time are centred on the line, close to it, and the symbol is exactly as wide as its line. Other options of relations (Ti*k*Z options) work as in a graph. Sizes are in em, so the symbols follow the font size; the sizes are the style `inline`, e.g. `\dcrset{inline/.append style={dcr/relation width=0.08em}}`, the distance of guard and time from the line `inline label distance` (default 0.12em). A symbol behaves like `\rightarrow`, and with a guard or time like `\xrightarrow`: in math it is a relation (same spacing, line breaks), in sub- and superscripts it shrinks with its guard and time ($X_{A \to B}$); in text, `A \dcrcondition B` is spaced as `A $\rightarrow$ B`, with word spaces (no space is added before punctuation: `A \dcrcondition.`). For `>` and `<` in guards, use `\usepackage[T1]{fontenc}` (with the old default encoding OT1 they print as other characters).

The format of guards on lines: `\dcrguardformat`; of the negated guard of a logical include in the classic and coopis23 notations: `\dcrnotguard`.

A Compatibility with the DCR portal and DCR-js

DCR-js XML. Read: events (label = `description`, roles, position and size, marking), nestings, subprocesses, relations of all types DCR-js has, their routes (with `waypoints`). DCR-js stores no guards, times, event types or colours; a graph imported into DCR-js from the portal has lost them (pre-conditions and logical includes become their two relations). Unknown elements and relation types are skipped with a warning.

The rest of this appendix is about DCR Solutions XML.

Read from the file. Events: label, roles, receivers (`readRoles`), position, colours (background, border, text), marking (executed, pending, included), type (data, computation, DMN, global, form, subgraph). Collections: nestings, subprocesses, forms, subgraphs with their own graph; their boxes

are computed as in the portal. Relations: condition, pre-condition, milestone, response, include, exclude, logical include, no-response, value (update), spawn; their guards and times; the routes of lines (only with the option `waypoints`, for DCR-js files).

Not reproduced. The portal's routing of lines (it draws some lines at right angles, the package draws straight lines and spreads out lines that would overlap); the order of parallel lines between two events; the value assigned by an update relation; data types, expressions and everything that only concerns the execution of the graph.

Sizes. Portal boxes are 115×125 px; the package draws them as 2.6×2.8 cm (its default box), the positions scaled accordingly.